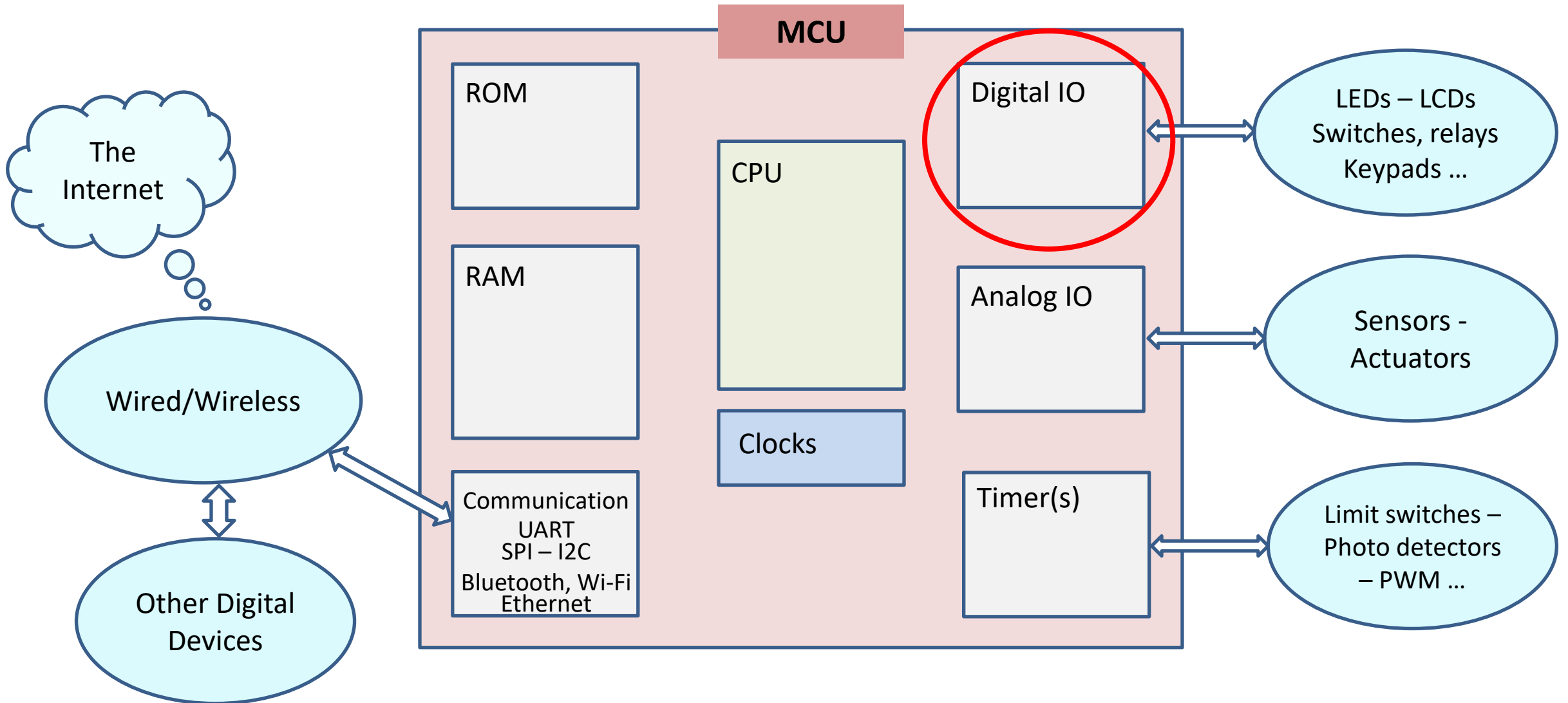


CSE211s Introduction to Embedded Systems

Spring 2026

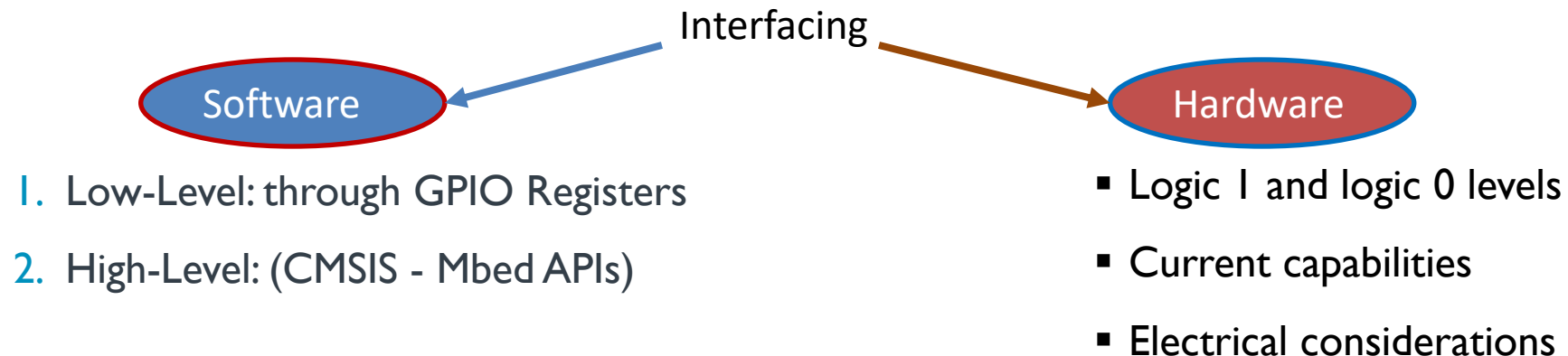
General Purpose IO (GPIO)

Typical Microcontroller Components



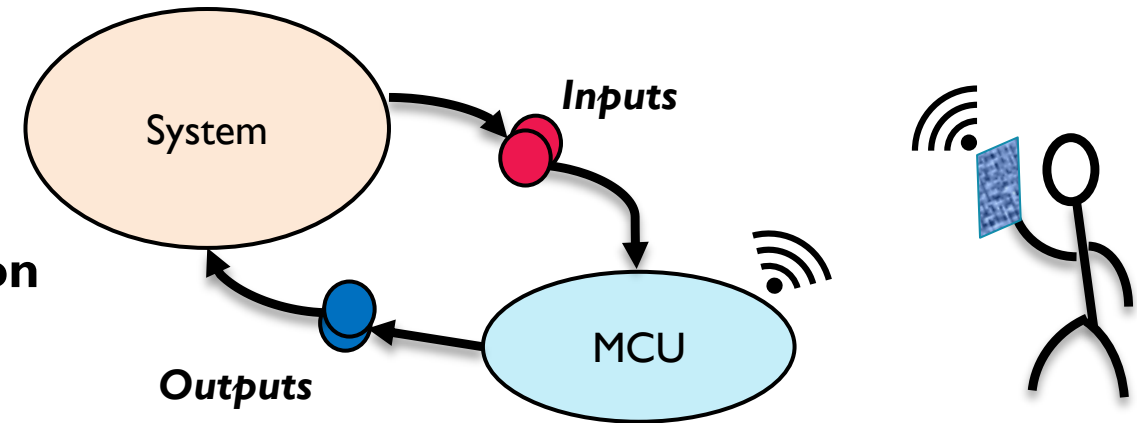
Lecture Topics

- Peripheral Modules
- Example Digital IOs
- GPIO Peripheral Module: Features — Initialization & Configuration — Operation
- GPIO Peripheral Module Registers, Control/Status – Data, addressing and registers-fields settings
- Example digital GPIO Module: STM32F401RE MCU
 - *Features: Ports, bits, and pins*
 - *Initialization & Configuration: Clock gating – Pin-Multiplexing – Bit direction*
 - *Operation: Input – Output – Bit set/reset*



Typical Embedded Systems

- **Embedded systems typically:**
 1. **Input data/measurement collected from the surrounding:**
 2. **Process the input by the integrated application code to see what to do.**
 3. **Output control to influence the surrounding.**
 4. **Keep the operator informed and in control.**



The MCU is interfaced to the sensors, actuators, operator, other MCUs ... through **peripheral modules** integrated within the MCU chip.

IO Peripheral Modules

IO operations are done through integrated **IO peripheral modules**.

Peripheral modules can be:

1. Digital:

- A single bit (from a switch/button) or a collection of bits forming one entity e.g., a byte
- A stream of bits transmitted or received sequentially on a serial link (wired/wireless)

2. Analog:

- Analog input from a sensor e.g., temperature sensor requires analog to digital conversion ADC (ATD)
- Analog output e.g., to generate audio signal requires digital to analog conversion DAC (DTA)

The embedded system developer should learn the requirements, both **hardware** and **software**, to use a peripheral module.

Viewing a Peripheral Module

- A peripheral module usually has a group of registers for configuring the module and for data transfer. These registers are viewed by the application software as within the memory map (memory-mapped IO), and categorized as CONTROL+STATUS, and DATA registers.
- A peripheral modules must be configured before being used by the application. Configuring a peripheral module is to agree upon the way of doing future operations.

Configuration is done by writing bits in specific fields in **Control Registers** within the module.

It is done by the startup code before passing control to the application.

Control registers are also used to control module's operation.

**CONTROL
REGISTERS**

**STATUS
REGISTERS**

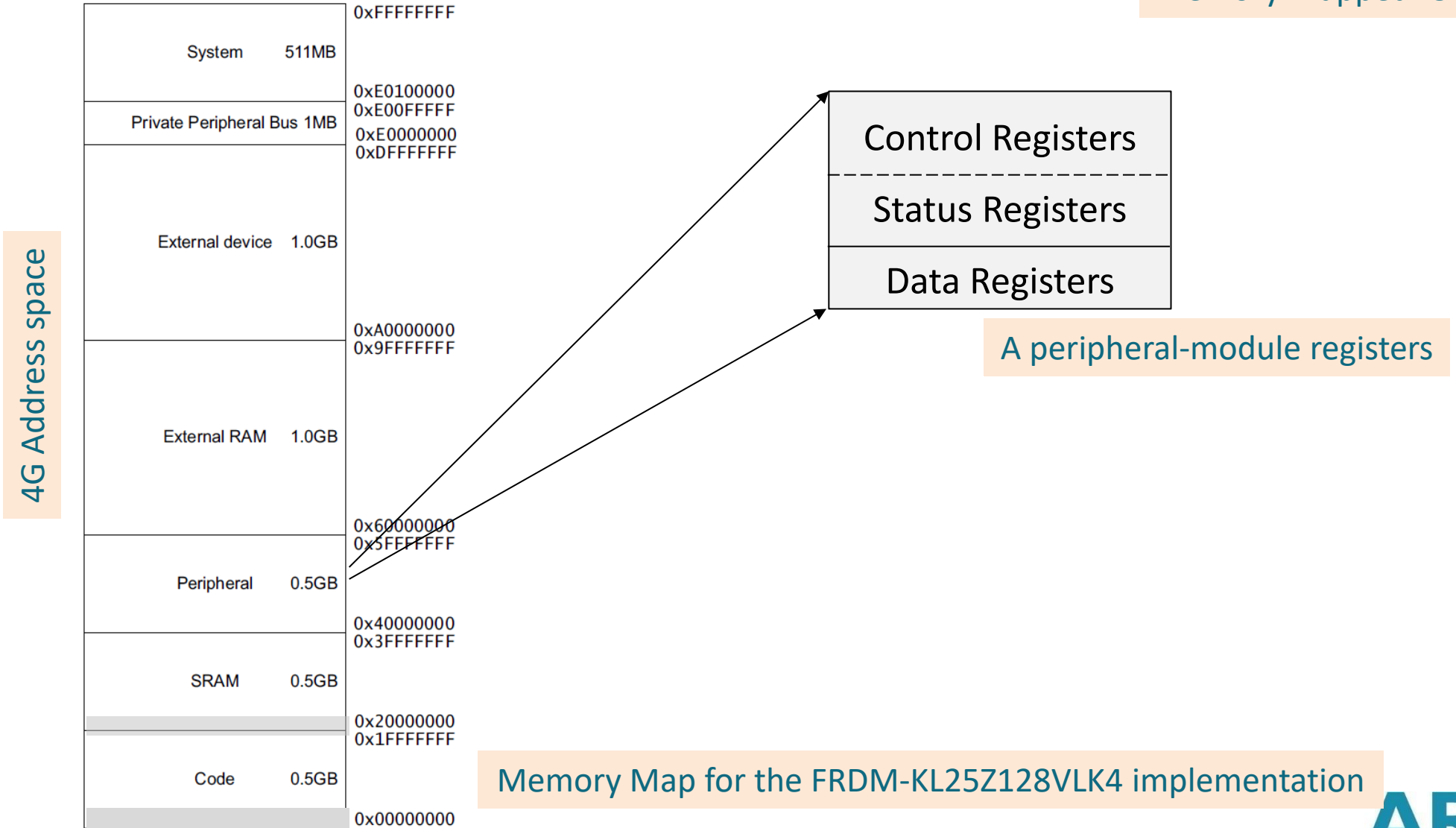
The status of the peripheral module is read from bits in specific fields in **Control+Status Registers** within the module.

**DATA
REGISTERS**

Data are read from and written to **Data Registers** within the module.

Addressing Peripheral Module Registers

Memory Mapped IO



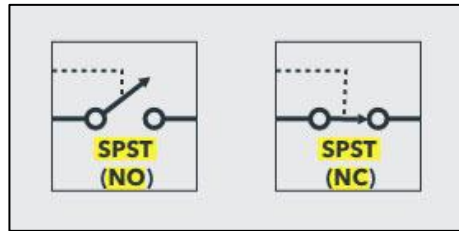
General Purpose IO

GPIO pins allow the microcontroller to:

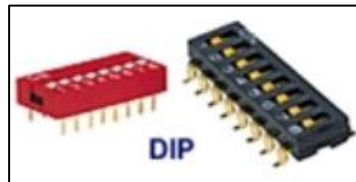
- Read digital signals from switches, buttons ...
- Output digital signals to LEDs, relays ...
- Connect to an internal timer, a serial communication module ... (alternate function)
- Connect to an analog signal

A pin that will carry a digital signal can be configured with different electrical characteristics
e.g., push-pull, open collector/drain

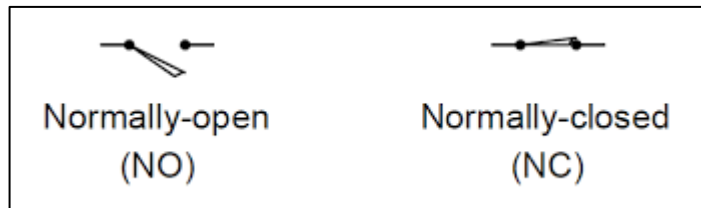
Example Digital Input Devices



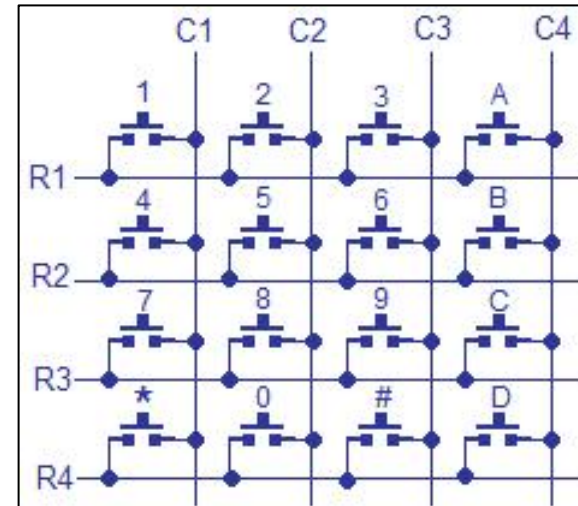
Single-Pole Single-Throw



Switches



Limit switch

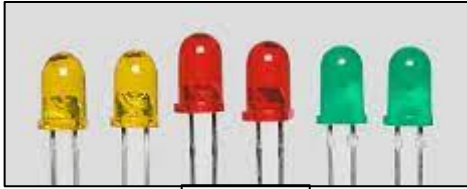


Keypad

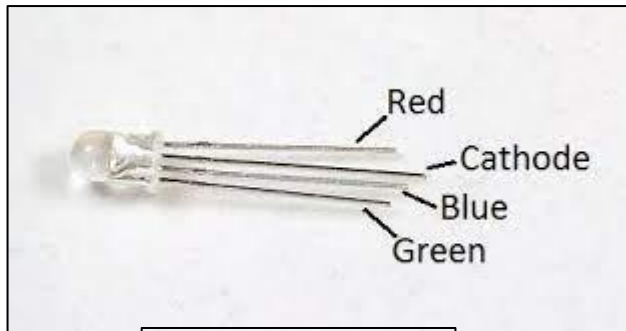


Touch slide

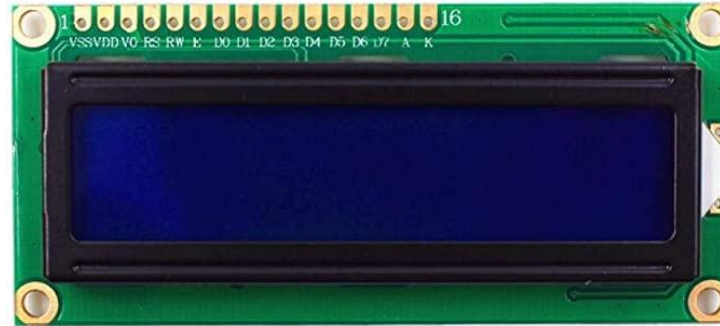
Example Digital Output Devices



LEDs



Tricolor LED



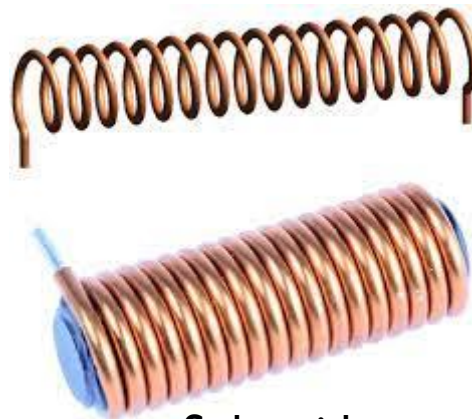
LCD



LED screen



Relays



Solenoid

Digital IO Considerations

Several points must be clear before using GPIO in a microcontroller:

- The available number of GPIO pins and ports

HARDWARE

Circuit interfacing to port pins:

- Acceptable voltage levels for logical 1 and 0
- Current sourcing and sinking capabilities
- Need for pull-up or pull-down resistors

SOFTWARE

1. Initialization and configuration:

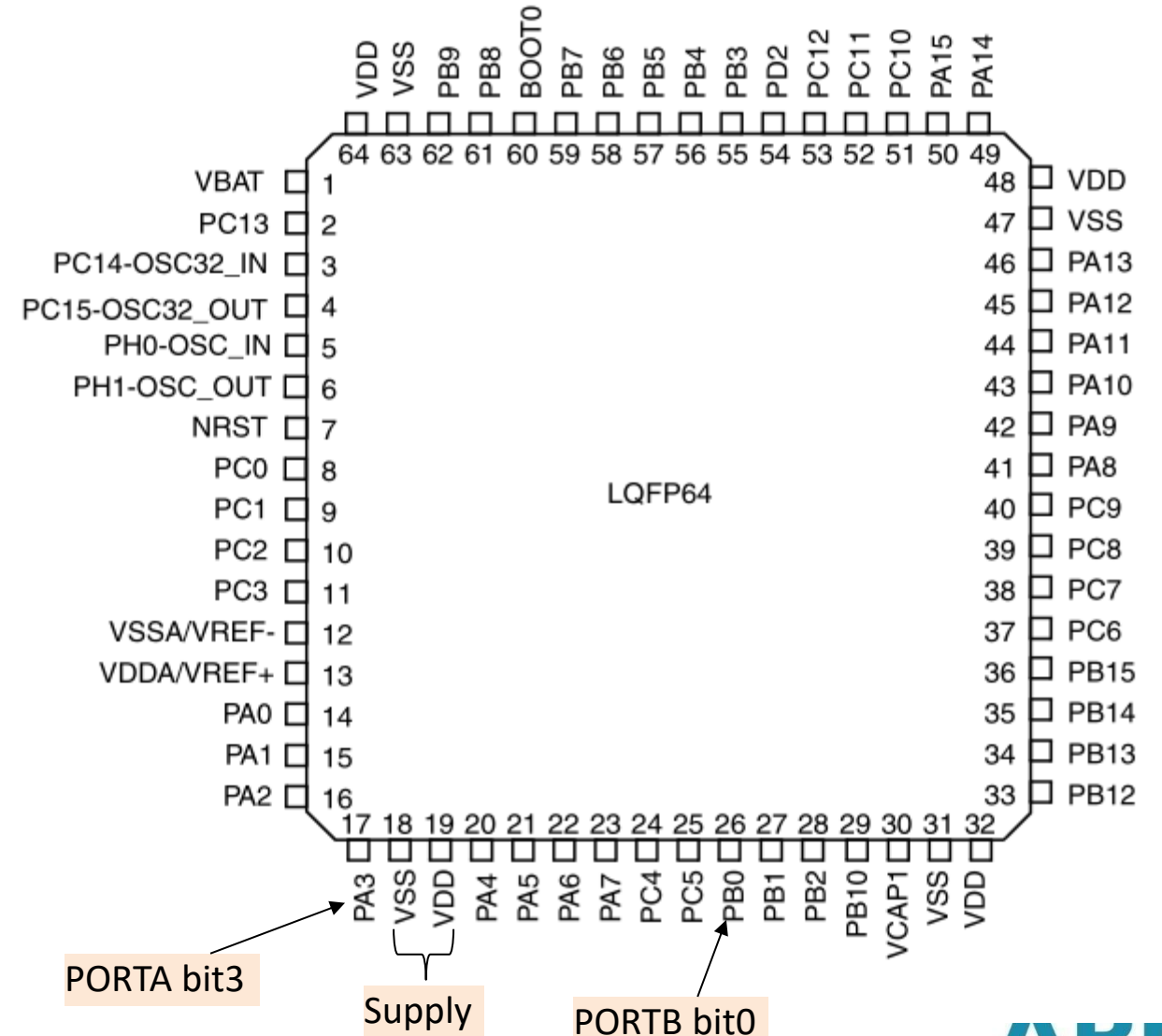
- Connecting port bits to MCU pins
- Defining data direction: (input or output)
 - Is direction configurable?
 - Is it defined for the port as a whole or on a bit basis?
 - What can we do with a port bit:? ... send value, set, clear, or toggle

2. Operation: How to read or write a pin value

STM32F401RE Pinouts (Example: 64-pin package)



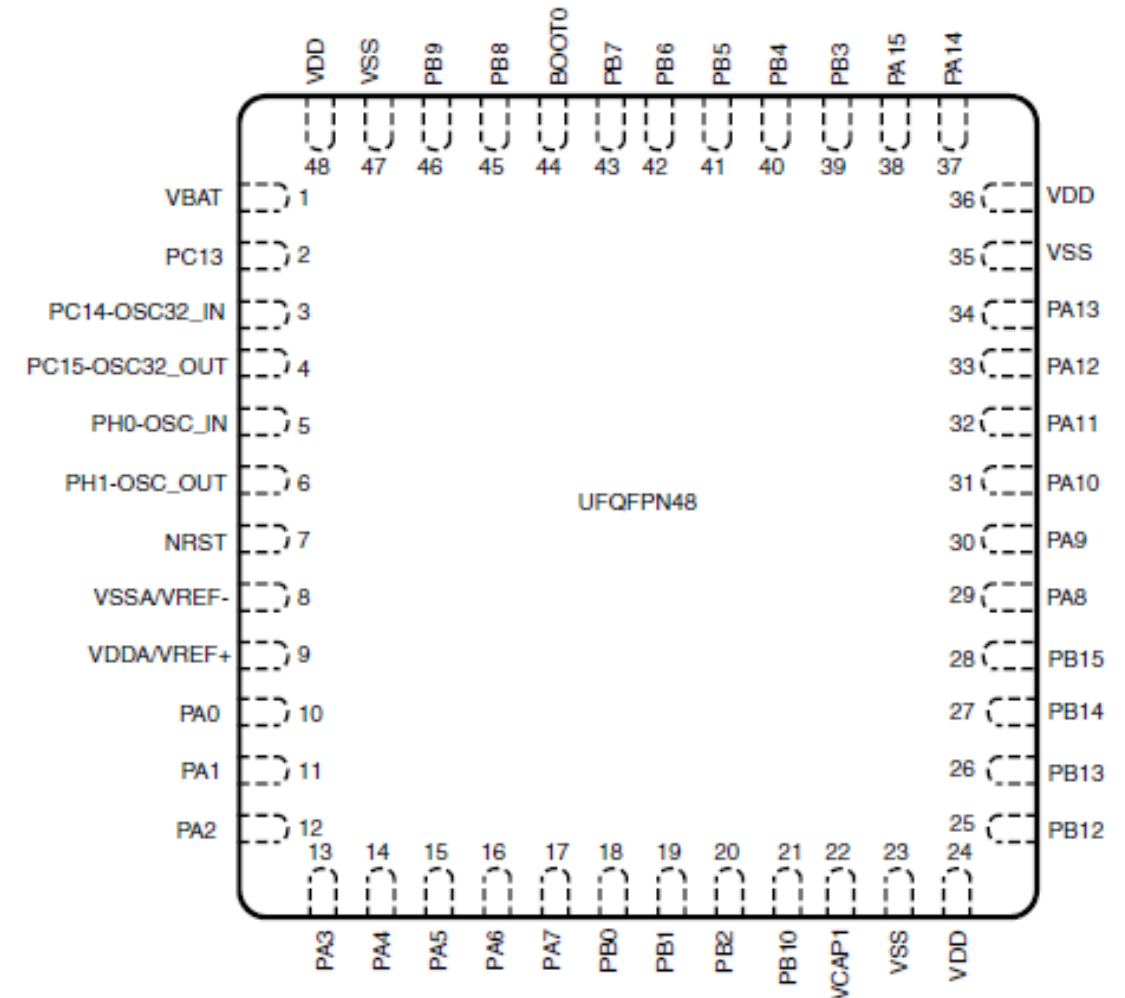
Low Profile Quad Flat (LPQF) Package



STM32F401RE Pinouts (Example: 48-pin package)

The number of IO pins is often fewer than the number of signals from on-chip peripheral modules especially in smaller packages.

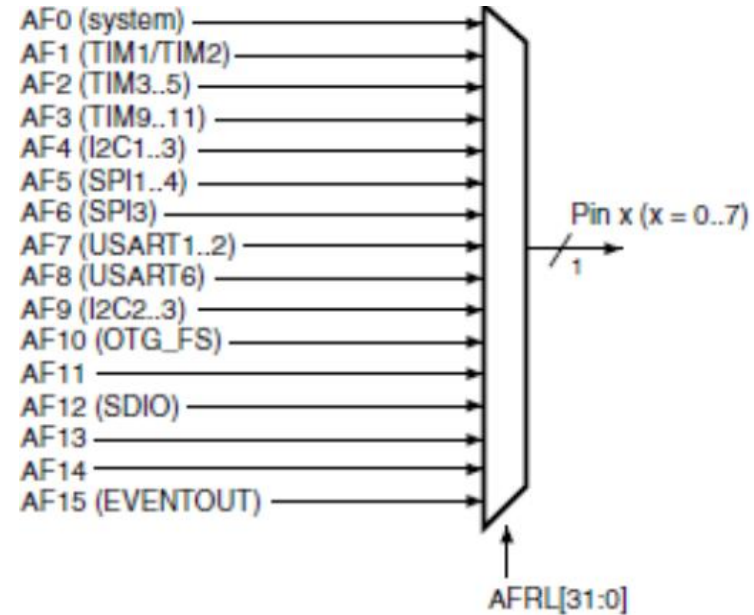
Therefore, there is a need for **pin multiplexing**.



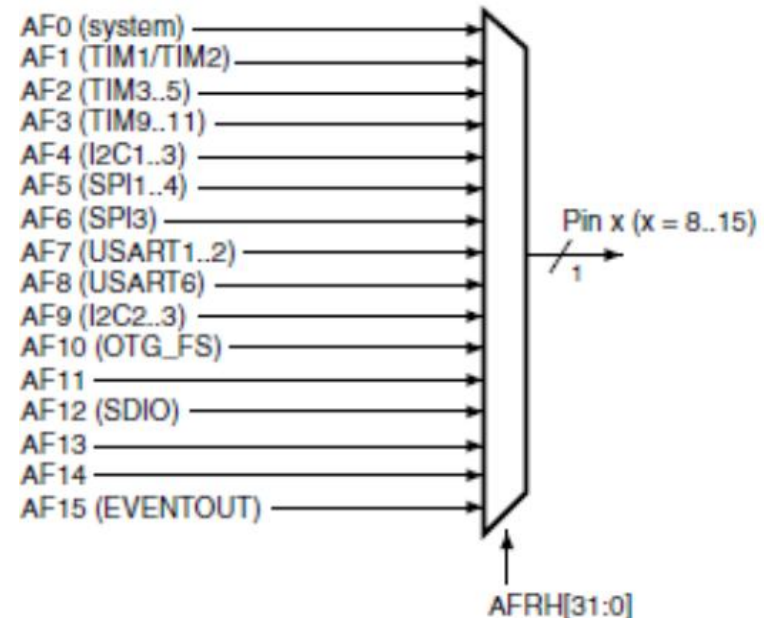
Pin Multiplexing

- Each pin is internally connected to a multiplexer to
 - Allow each pin connect to several on-chip modules to perform several functions
 - Optimize functionality for small packages
 - Connect only one peripheral's signal to an IO pin at a time.
- The pin multiplexer in the [STM32F401RE](#) MCU has sixteen alternate function inputs (AF0 to AF15) that can be configured through two register:
 - GPIOx_AFRL (for pin 0 to 7) and
 - GPIOx_AFRH (for pin 8 to 15)

If you are using a pin as a GPIO, you typically do not need to configure the pin multiplexer explicitly as it is the default.



For pins 8 to 15, the GPIOx_AFRH[31:0] register selects the dedicated alternate function



Alternate Functions & Additional Functions

GPIO:

Right after reset, the alternate functions are not active, and the I/O ports are configured for GPIO in input floating mode.



For system safety

Alternate Functions:

A pin can be used for alternate function e.g., UART, SPI, I2C, etc. The function is selected through GPIOx_AFRL and GPIOx_AFRH registers that control the pin multiplexer.

Additional Functions:

Functions directly selected/enabled through peripheral registers.

- Pull-up/Pull-down resistors: Configure internal resistors for input pins.
- External Interrupts (EXTI): Any GPIO pin can trigger an interrupt.
- Analog Input Mode: Pins can be used for ADC.
- Open-Drain Configuration: Allows a GPIO pin to behave like an open-drain output.

STM3240IRE GPIO Ports

- GPIOs split into groups, called ports.
- The STM32F40IRE MCU has eight GPIO ports named **PORTA - PORTB - PORTC - PORTD - PORTE – PORTH** each has 16 bits.
- There are not enough pins on the MCU package to support all these bits, hence:

Not all port data bits are implemented.

The MCU's data sheet pin-out section describes which port bits are implemented.

Quantity depends on package pin count, which differ from one MCU to another.

- To reduce power consumption, clocks are turned off for most peripherals after reset.
 - To use a peripheral or a resource, clock must be enabled for it individually.

GPIO INTERFACING

1- Hardware

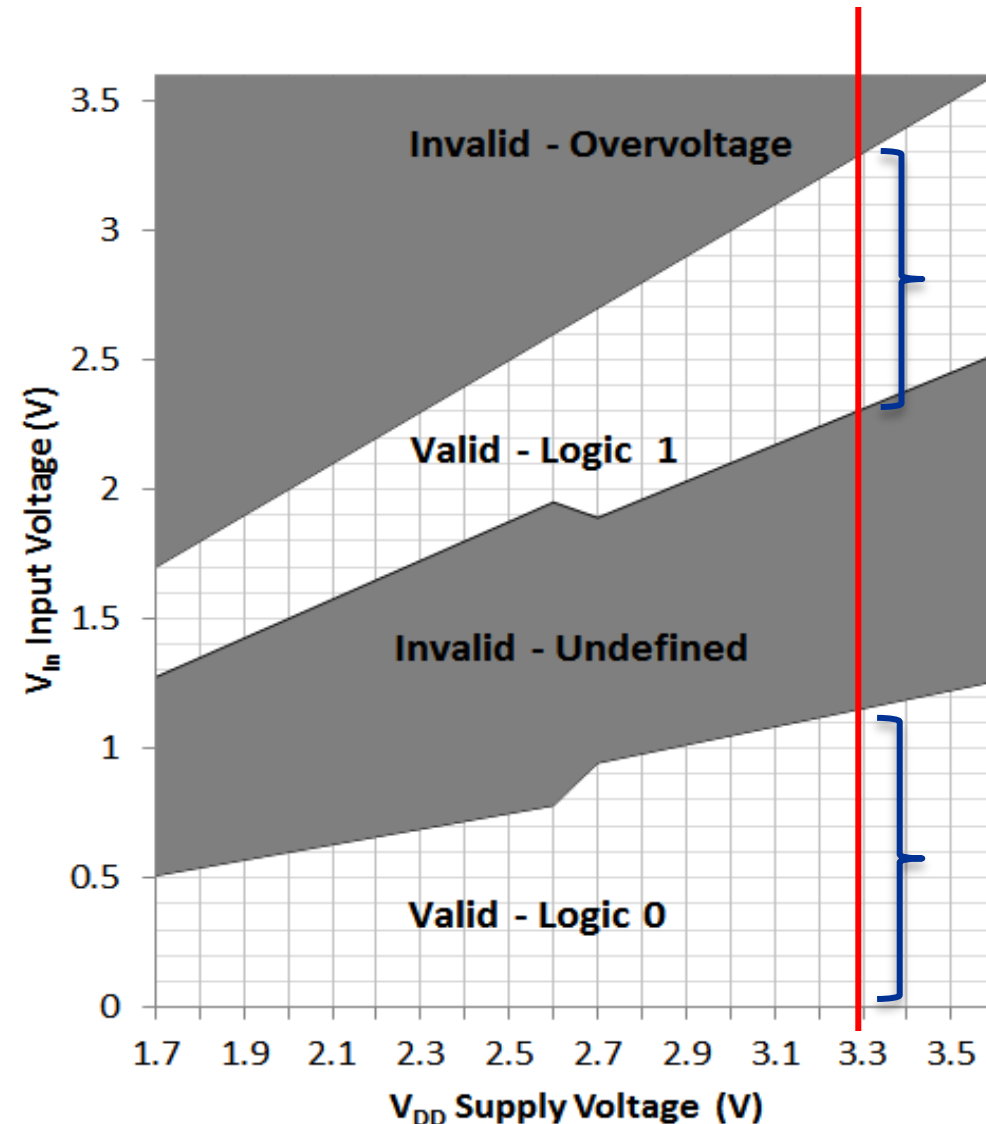
Voltages and Logic Values

- In digital devices, logical values : ' 1' or '0', are represented as electrical voltages.
- Different devices may use different voltages to represent a logic value.
For example, the external pins of the stm32F401RE use
 - 3.3 volts for logic '1', and
 - 0 volts for logic '0 '
- Different logic can, however, have different meanings in different contexts depending on the interpretation we give to different voltage levels

Logic	Voltage	Boolean	Circuit	Switch
'1'	3.3V	True	Closed	On
'0'	0V	False	Open	Off

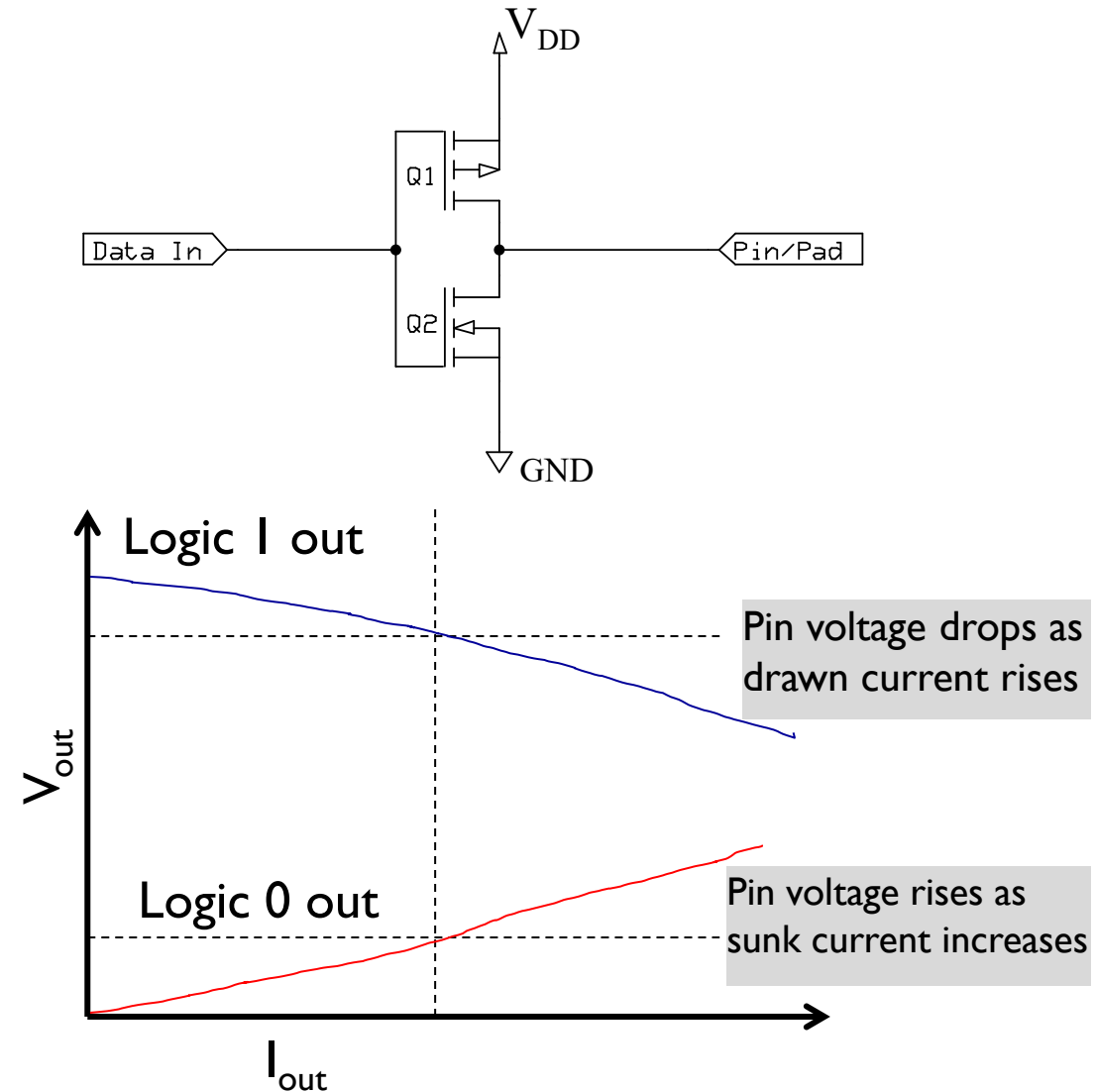
Inputs: What's a 1 and a 0?

- Input signal's value is determined by voltage.
- Input threshold voltages depend on supply voltage V_{DD}
- Exceeding V_{DD} or GND may damage chip
- MCU chip dependent
- In the graph with $V_{DD} = 3.3V$ (red line)
 - A valid logical 0 range is 0V to 1.3V.
 - A valid logical 1 range is 2.6V to 3.3V.



Effect of the load current on the pin's logical output

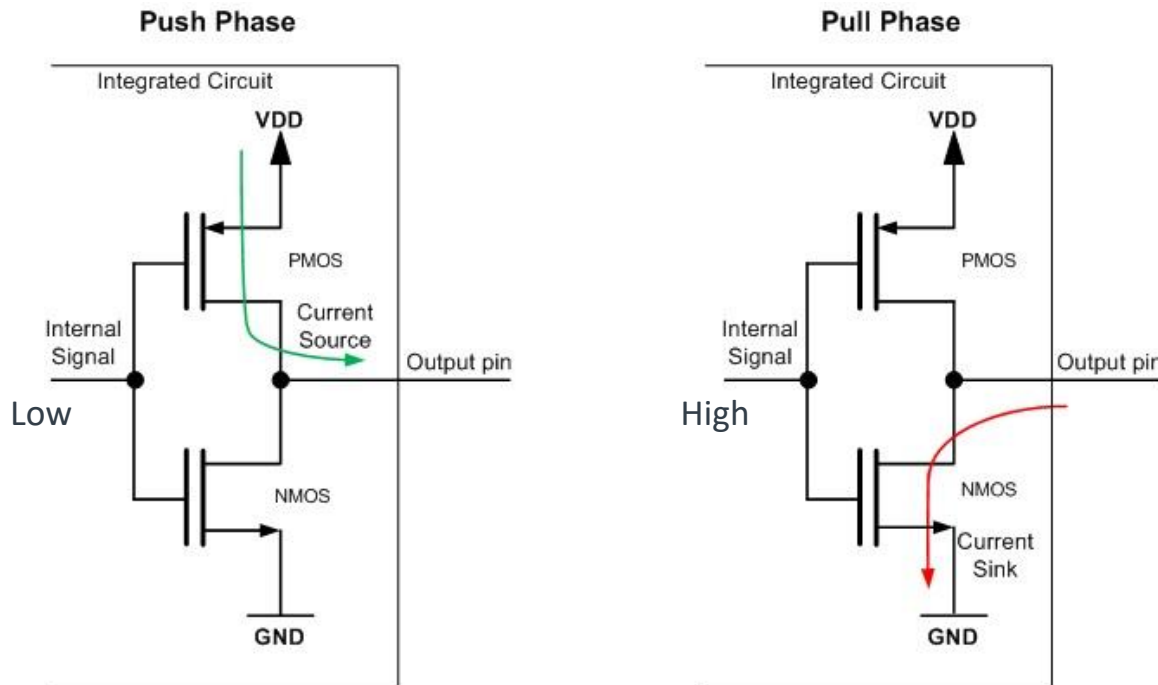
- Nominal output voltages specified when current $< 5\text{ mA}$ (or 18 mA for high-drive pads) and $V_{DD} > 2.7\text{ V}$:
 - 1: minimum $(V_{DD} - 0.5\text{ V})$ to maximum V_{DD}
 - 0: minimum 0 to maximum 0.5 V
- Pin's output voltage depends on the current drawn from or sunk by the pin. (see the plot)
 - Need to consider source-to-drain resistance in the transistor
- MCU chip dependent



Output Pin Connection Modes

Push-Pull

Connect the pin either to V or GND



Pin is connected to V_{DD}
Sourcing current

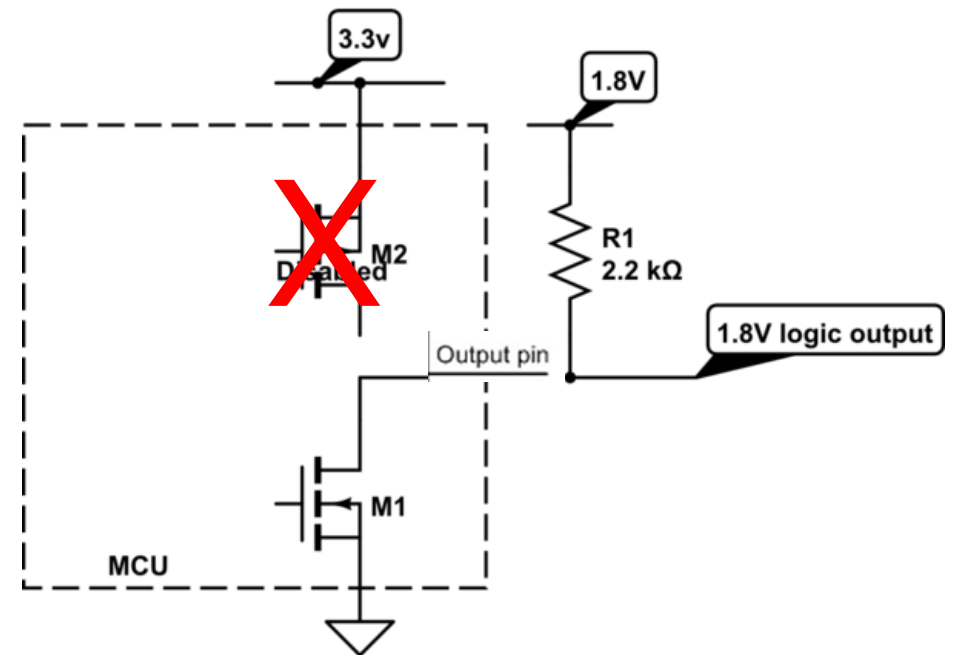
Pin is connected to GND
Sinking current

Voltage level and current
are from MCU chip

Open Drain

Connect the pin to GND or leave float

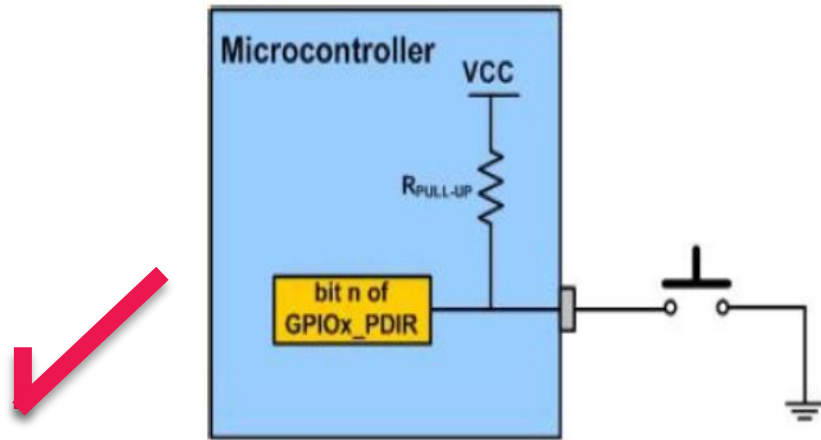
Simple interface to 1.8V logic (slow)



Voltage level and current are provided by an outside source

Software programmable (selectable), push-pull or open-drain

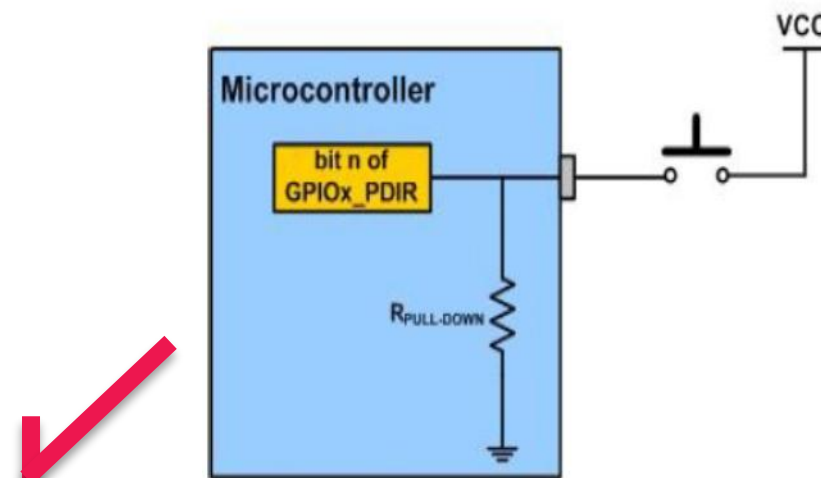
Input Pin Connection Modes



Using pull-up resistor

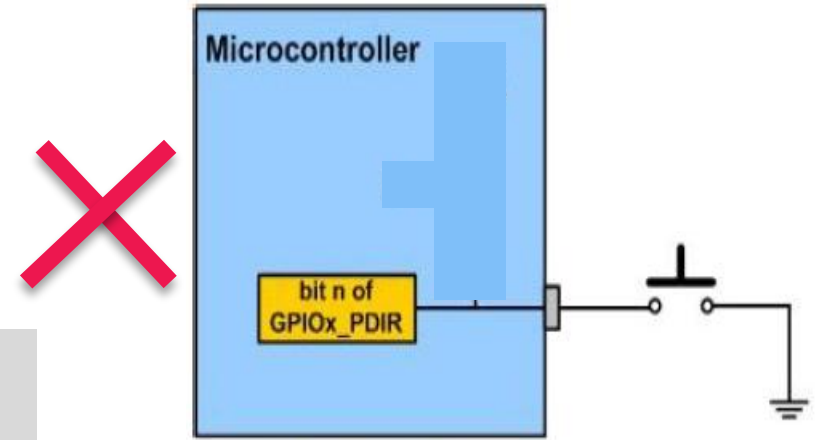
Never leave an input pin floating.
Use internal (or external) pull-up or pull-down resistor.

This is to ensure input signal voltage is pulled to correct value when switch is open (high-impedance).

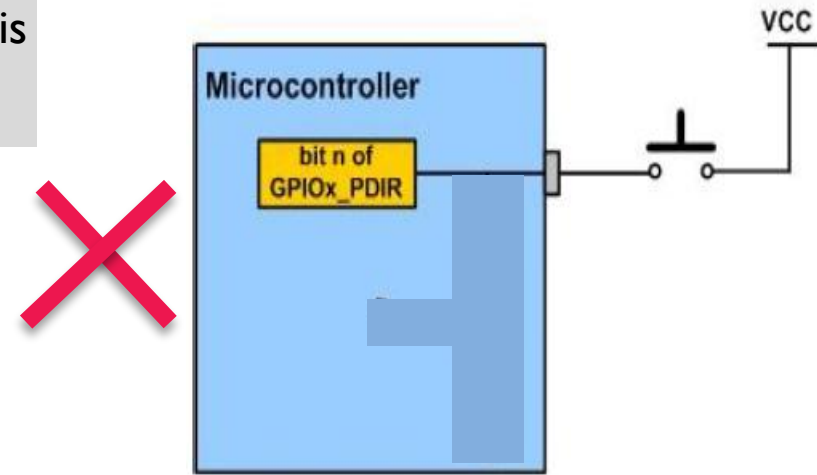


Using pull-down resistor

Software programmable (selectable)



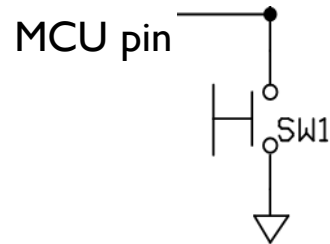
Floating input



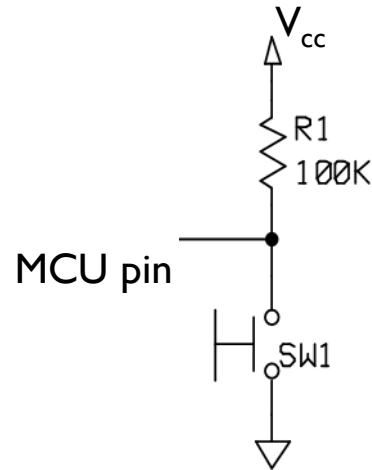
Floating input

IO Connection Examples

Input Connections



Using Internal Pullup



Using External Pullup

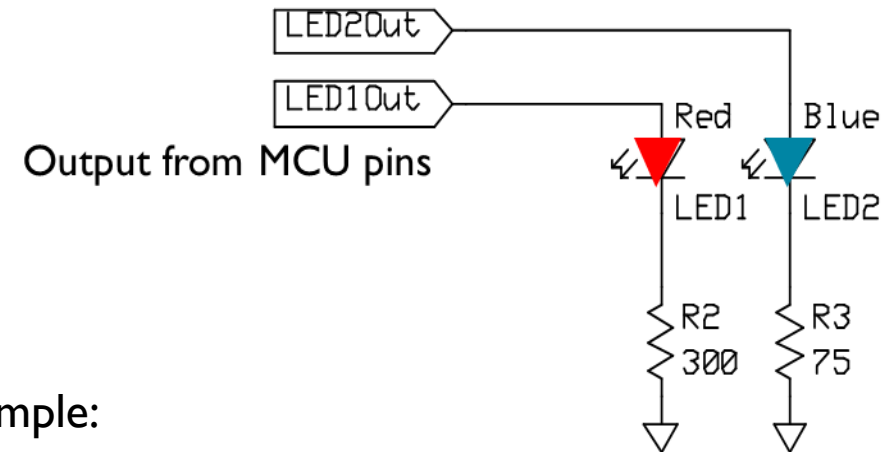
Example:

If input SW1 is Normally Open (NO):

- Pressed → input to MCU pin is V_L → logical zero
- Released → input to MCU pin is V_{cc} → logical one

A pin connected with external pullup must be able to **sink** the current $\frac{V_{cc}-V_L}{R1}$ when the switch is released.

Output with current limiting resistor



Example:

Output LEDs:

- MCU output pin is low → LED is off
- MCU output pin is high → LED is on

The MCU pin must be able to **source** the current I_{LED_i} for suitable illumination.

The resistances are chosen to satisfy

$$I_{LED_i} = \frac{V_H - V_{LED_i}}{R_i}$$

Output Example: Driving LEDs

- Need to limit current to a value which is
 - suitable for LED illumination
 - safe for the MCU pin driver
- Use current-limiting resistor

$$R_i = \frac{V_{DD} - V_{LED_i}}{I_{LED_i}}$$

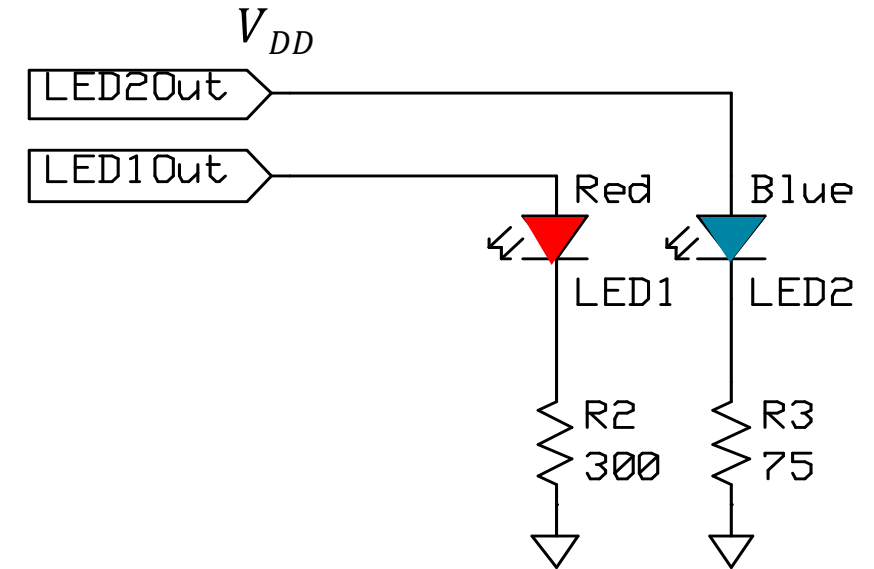
Example:

If from the LED data sheet for clear brightness I_{LED_i} should be 4 mA, and V_{LED_i} depends on type of LED (mainly color)

Red: ~1.8V

Blue: ~2.7V

- Solve for R_i given $V_{DD} \approx 3.0V$
 - Red: 300 Ω
 - Blue: 75 Ω

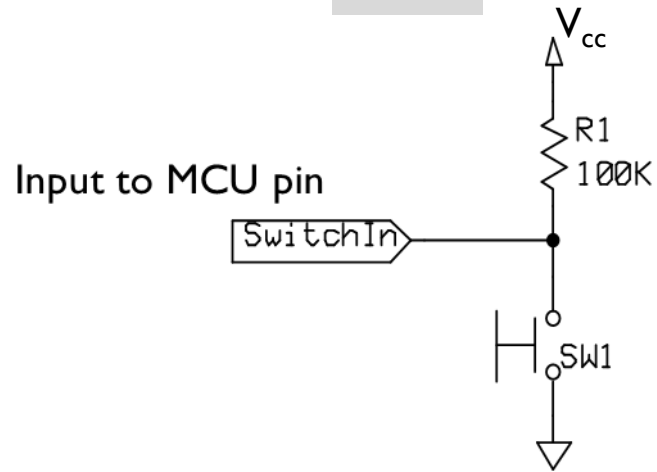


From the MCU data sheet you may find that:

- The maximum output current from a single pin is 25 mA
- The total maximum current from all pins is 120 mA

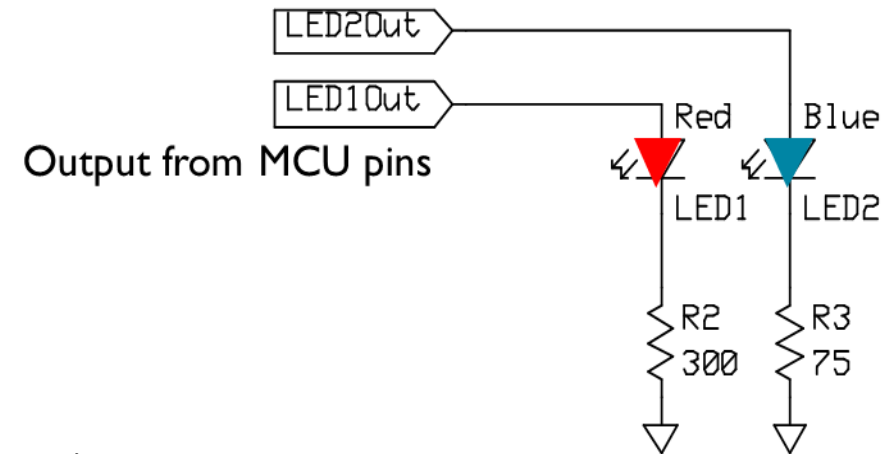
Homework

Input



Change the input circuit shown to cause logical one when the key is pressed and zero if released.

Output

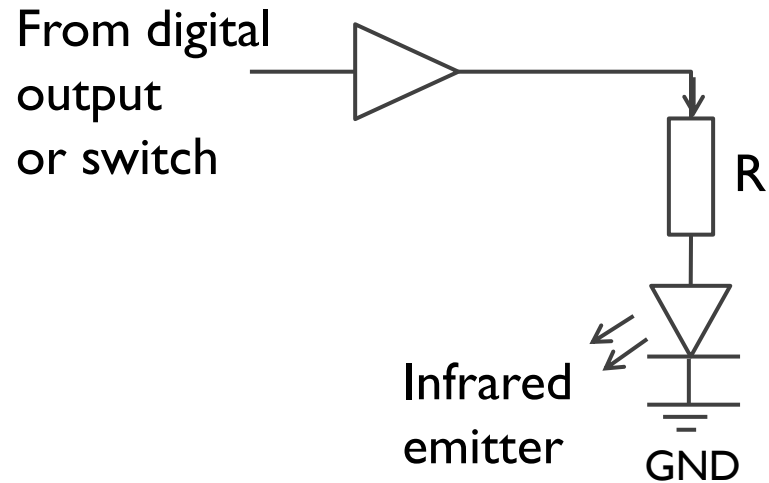


Change the output circuit shown to turn on the LED by a logical zero on the pin and turn it off by a logical one.

Assume the pin is to drive a load that needs a current higher than the pin's capability to source (or sink). Show how to interface the pin to the load using transistors in two cases:

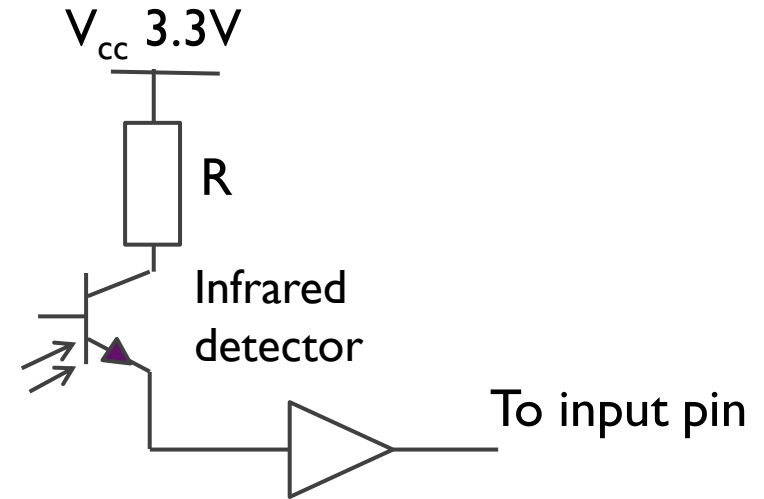
1. Sourcing the current
2. Sinking the current

Infrared Wireless Link



- Infrared emitter (LED)

- A light-emitting diode that emits invisible infrared (IR) when conducting e.g. when connected to digital output:
- '1' – IR emitted (Brightness above a threshold)
- '0' – no IR emitted (Brightness below a threshold)



- Infrared detector

- Converts light into either current or voltage e.g. when connected to digital input:
- '1' – IR received (Brightness above a threshold)
- '0' – no IR received (Brightness below a threshold)